

Distributed Systems

Lecture15: Message-oriented Communication

Mohammad Reza Shakournia

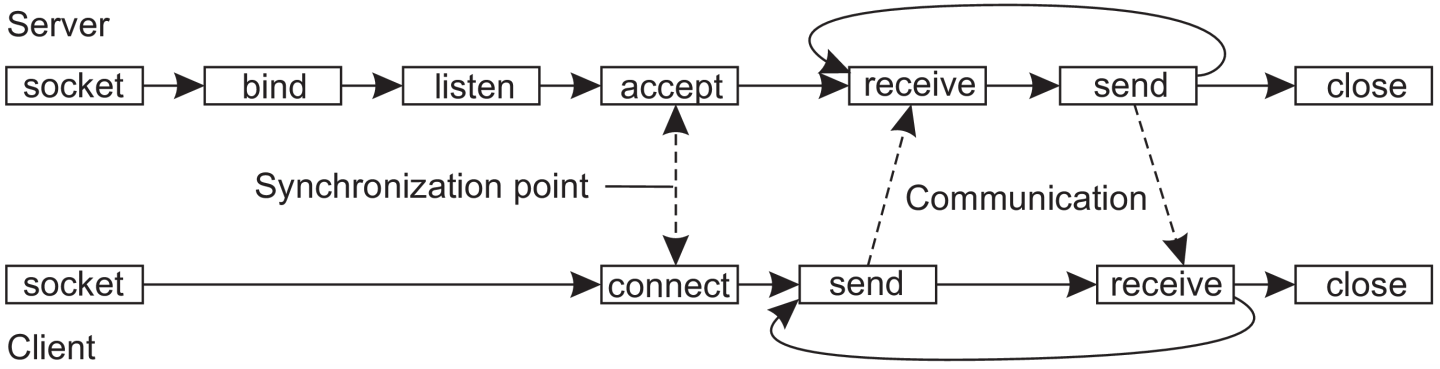
shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Transient Messaging: Sockets

- Berkeley Socket Interface:

Operation	Description
`socket`	Create a new communication end point
`bind`	Attach a local address to a socket
`listen`	Tell operating system what the maximum number of pending connection requests should be
`accept`	Block caller until a connection request arrives
`connect`	Actively attempt to establish a connection
`send`	Send some data over the connection
`receive`	Receive some data over the connection
`close`	Release the connection



Sockets: Python Code

- Server:

```
1  from socket  import *
2
3  class Server:
4    def run(self):
5      s = socket(AF_INET, SOCK_STREAM)
6      s.bind((HOST, PORT))
7      s.listen(1)
8      (conn, addr) = s.accept()  # returns new socket and addr. client
9      while True:                # forever
10       data = conn.recv(1024)   # receive data from client
11       if not data: break       # stop if client stopped
12       conn.send(data+b"*")     # return sent data plus an "*"
13     conn.close()               # close the connection
```

Sockets: Python Code

- Client:

```
1  from socket  import *
2
3  class Client:
4    def run(self):
5      s = socket(AF_INET, SOCK_STREAM)
6      s.connect((HOST, PORT)) # connect to server (block until accepted)
7      s.send(b"Hello, world") # send same data
8      data = s.recv(1024)     # receive the response
9      print(data)             # print what you received
10     s.send(b"")             # tell the server to close
11     s.close()               # close the connection
```

Making Sockets Easier to Work with

- Sockets are rather low level and programming mistakes are easily made. However, the way that they are used is often the same (such as in a client-server setting).
- **Alternative: ZeroMQ**
  - Provides a higher level of expression by pairing sockets: one for sending messages at process P and a corresponding one at process Q for receiving messages. All communication is asynchronous.
- **Three Patterns:**
  - Request-reply
  - Publish-subscribe
  - Pipeline

ZeroMQ: Request-reply

```
1  import zmq
2
3  def server():
4    context = zmq.Context()
5    socket  = context.socket(zmq.REP)       # create reply socket
6    socket.bind("tcp://*:12345")            # bind socket to address
7
8    while True:
9      message = socket.recv()               # wait for incoming message
10     if not "STOP" in str(message):        # if not to stop...
11       reply = str(message.decode())+'*'   # append "*" to message
12       socket.send(reply.encode())         # send it away (encoded)
13     else:
14       break                               # break out of loop and end
15
16 def client():
17   context = zmq.Context()
18   socket  = context.socket(zmq.REQ)       # create request socket
19
20   socket.connect("tcp://localhost:12345") # block until connected
21   socket.send(b"Hello world")             # send message
22   message = socket.recv()                 # block until response
23   socket.send(b"STOP")                    # tell server to stop
24   print(message.decode())                 # print result
```

ZeroMQ: Publish-Subscribe

```
1  import multiprocessing
2  import zmq, time
3
4  def server():
5    context = zmq.Context()
6    socket = context.socket(zmq.PUB)          # create a publisher socket
7    socket.bind("tcp://*:12345")              # bind socket to the address
8    while True:
9      time.sleep(5)                           # wait every 5 seconds
10     t = "TIME " + time.asctime()
11     socket.send(t.encode())                 # publish the current time
12
13 def client():
14   context = zmq.Context()
15   socket = context.socket(zmq.SUB)          # create a subscriber socket
16   socket.connect("tcp://localhost:12345")   # connect to the server
17   socket.setsockopt(zmq.SUBSCRIBE, b"TIME") # subscribe to TIME messages
18
19   for i in range(5):       # Five iterations
20     time = socket.recv()   # receive a message related to subscription
21     print(time.decode())   # print the result
```

ZeroMQ: Pipeline

```
1  def producer():
2    context = zmq.Context()
3    socket  = context.socket(zmq.PUSH)      # create a push socket
4    socket.bind("tcp://127.0.0.1:12345")    # bind socket to address
5
6    while True:
7      workload = random.randint(1, 100)     # compute workload
8      socket.send(pickle.dumps(workload))   # send workload to worker
9      time.sleep(workload/NWORKERS)         # balance production by waiting
10
11 def worker(id):
12   context = zmq.Context()
13   socket  = context.socket(zmq.PULL)      # create a pull socket
14   socket.connect("tcp://localhost:12345") # connect to the producer
15
16   while True:
17     work = pickle.loads(socket.recv())     # receive work from a source
18     time.sleep(work)                       # pretend to work
```

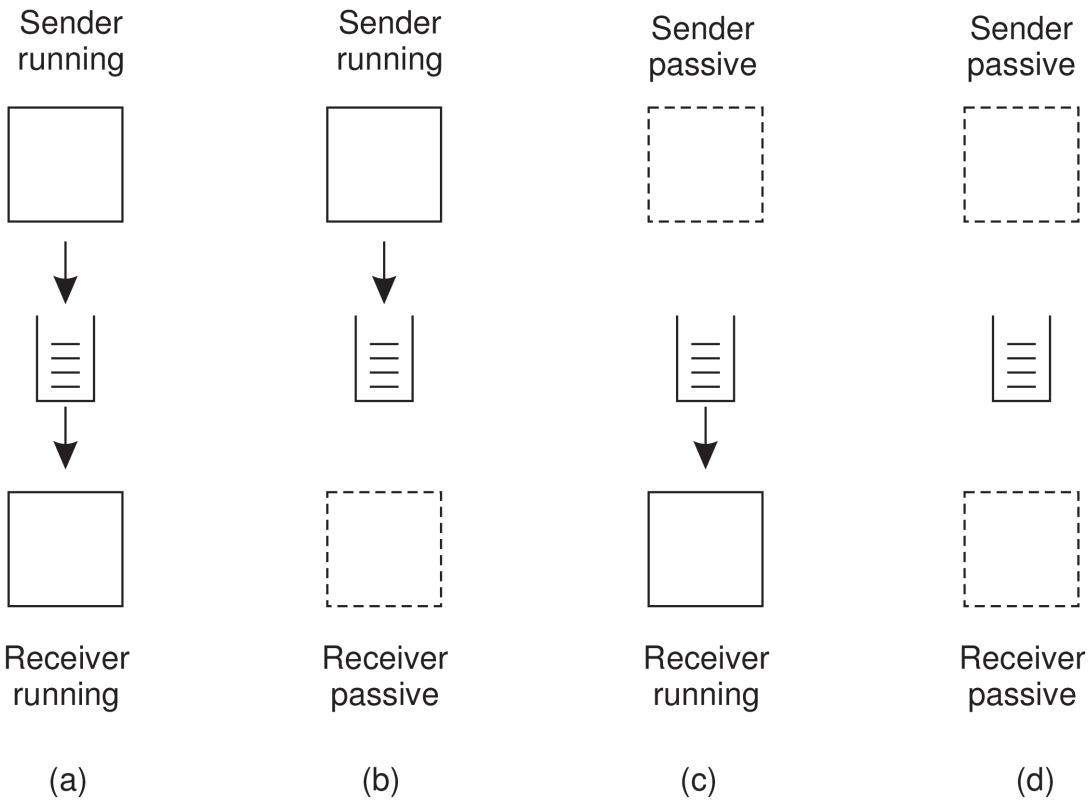
MPI: When Lots of Flexibility is Needed

- Representative operations:

Operation	Description
`MPI_BSEND`	Append outgoing message to a local send buffer
`MPI_SEND`	Send a message and wait until copied to local or remote buffer
`MPI_SSEND`	Send a message and wait until transmission starts
`MPI_SENDRECV`	Send a message and wait for reply
`MPI_ISEND`	Pass reference to outgoing message, and continue
`MPI_ISSEND`	Pass reference to outgoing message, and wait until receipt starts
`MPI_RECV`	Receive a message; block if there is none
`MPI_IRECV`	Check if there is an incoming message, but do not block

Queue-based Messaging

- Four possible combinations:



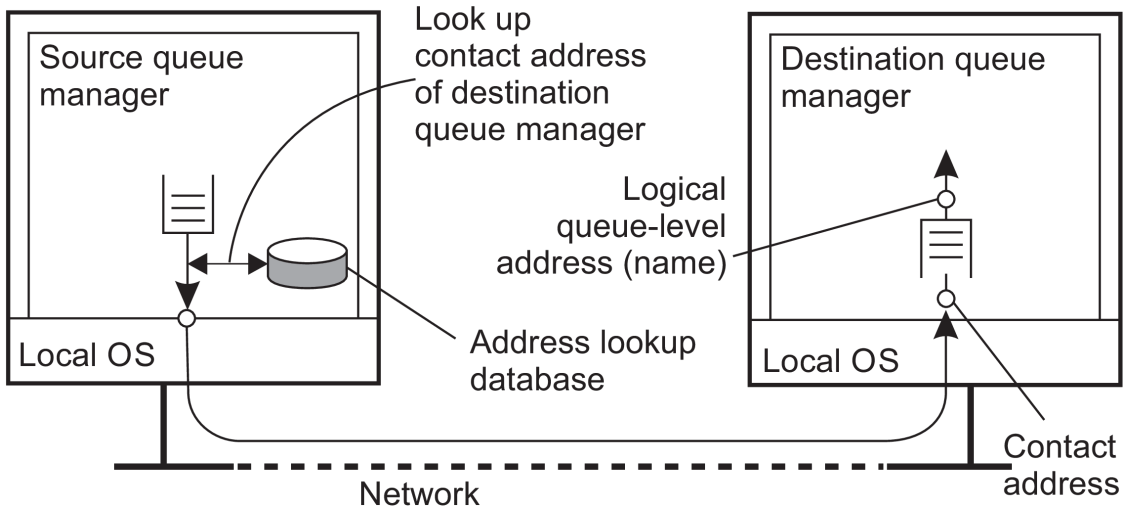
Message-oriented Middleware

- Asynchronous persistent communication through support of middleware-level queues. Queues correspond to buffers at communication servers.

Operation	Description
PUT	Append a message to a specified queue
GET	Block until the specified queue is nonempty, and remove the first message
POLL	Check a specified queue for messages, and remove the first. Never block
NOTIFY	Install a handler to be called when a message is put into the specified queue

General Model

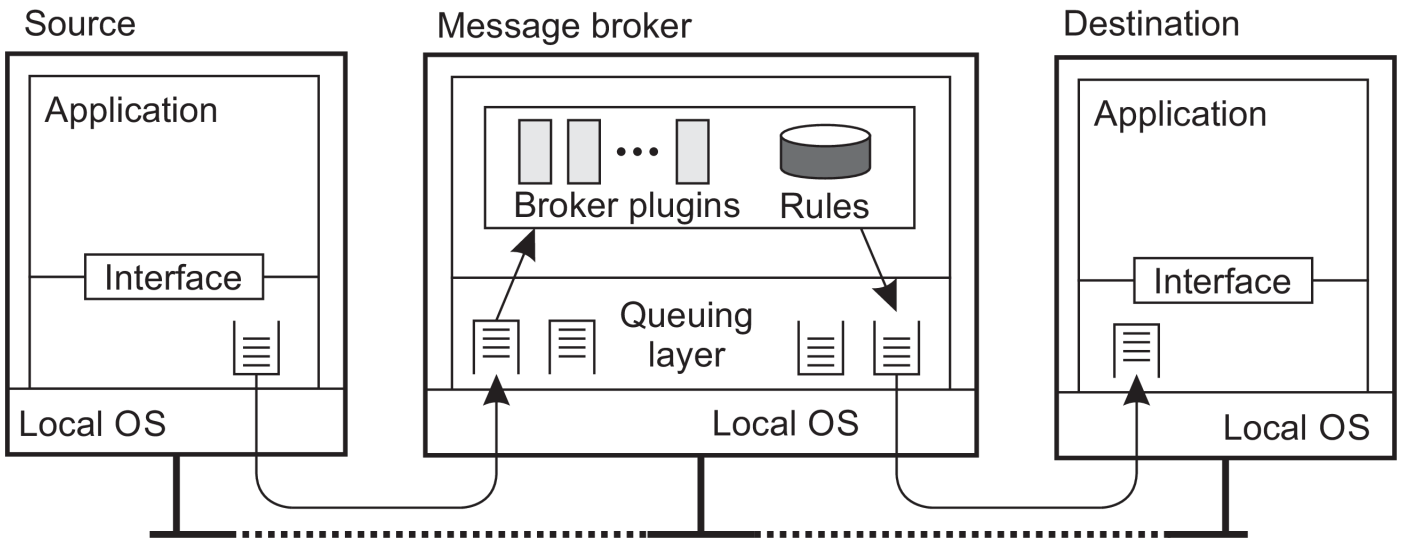
- **Queue managers:**
  - Queues are managed by queue managers. An application can put messages only into a local queue. Getting a message is possible by extracting it from a local queue only ⇒ queue managers need to route messages.
- **Routing**



Message Broker

- Message queuing systems assume a common messaging protocol: all applications agree on message format (i.e., structure and data representation).
- **Broker handles application heterogeneity in an MQ system**
  - Transforms incoming messages to target format
  - Very often acts as an application gateway
  - May provide subject-based routing capabilities (i.e., publish-subscribe capabilities)

Message Broker: General Architecture



Wrap-Up + What's Next

In the next lecture, we will discuss the topic about distributed system which are able to send data to multiple receivers known as Multicast Communication, including:

- Application-level Tree-based Multicasting,
- Flooding-based Multicasting,
- Gossip-based Data Dissemination.